

CS4523 Project Report

Deterministic Hash Sets vs. Randomized Bloom Filters for Set-Membership Testing

Date of Submission:09/21/2026

Project by :

220087R - Chathurangi K.L.

220419N - Navodi S.Y.A.C

Table of content

Table of content	2
Abstract	3
1.0 Introduction & Theoretical Background	3
2.0 Methodology & Setup	4
2.1 Real-world dataset composition	4
2.2 Detailed Experimental Procedure	5
2.2.1 Data Generation	5
2.2.2 The Hashing Mechanism	6
2.2.3 Hash Set (V1) Operation, Step by Step	7
2.2.4 Bloom Filter (V2) Operation, Step by Step	7
2.2.5 Benchmark Procedure	8
2.2.6 Orchestration	9
2.3 Environment Setup	10
2.4 Benchmark execution	10
3.0 Results and Discussion	11
3.1 Memory Footprint	11
3.3 False-Positive Rate (FPR)	17
3.4 Parameter Sensitivity	19
4.0 Conclusion	23
4.1 Recommended use cases:	23
4.2 Practical Applications:	23

Abstract

This report presents an empirical comparison of two set-membership data structures: a deterministic, separate-chaining Hash Set (V1) and a randomized, bitarray-backed Bloom Filter (V2) built from scratch using Kirsch–Mitzenmacher double hashing over MurmurHash3. Both structures were benchmarked across synthetic alphanumeric key sets scaling from 10^3 to 10^6 elements and a real-world pool of phishing URLs drawn from three public datasets, along four axes: memory footprint, insertion/query latency, false-positive rate (FPR), and sensitivity to the Bloom filter's configuration parameters m (bit-array size) and k (hash-function count).

The central trade-off shown by the results is clear: at one million elements, the Bloom filter occupies roughly **1.14 MB** against the hash set's **229 MB**, a $\sim 200\times$ reduction in memory, at the cost of a small, precisely predictable false-positive rate (2.24% empirically at 52.8% fill, against a theoretical prediction of 2.16%) and the complete inability to retrieve or delete elements. Across every experiment, the Bloom filter's zero-false-negative guarantee held without a single exception, and its empirical false-positive rate tracked the closed-form prediction closely enough to validate the underlying probabilistic model [1]. The parameter-sensitivity sweep further confirmed that the analytically optimal hash-function count $k = (m/n) \cdot \ln 2$ sits at or immediately beside the empirically best-performing k in every configuration tested [1]. The overall conclusion is that the two structures are not competitors but complements: the hash set is the correct choice wherever exact answers or retrieval are required, while the Bloom filter is the correct choice wherever approximate membership at large scale, under a tight memory limit, is acceptable.

1.0 Introduction & Theoretical Background

Set-membership testing, which is deciding whether an element x belongs to a set S , is a primitive that appears throughout computing, from database indexes to network routers. This project contrasts two paradigms for answering that question:

- **The exact paradigm (Hash Set):** Every inserted element is stored explicitly, hashed to a bucket, and retained via separate chaining on collision. A query is answered by direct comparison against the stored keys, so the result is always correct but the structure's memory footprint necessarily grows with both the number of elements and the size of each element, since every key must be kept in full.
- **The probabilistic paradigm (Bloom Filter):** No element is ever stored. Instead, a fixed-size bit array of m bits is manipulated by k hash functions: insertion sets k bits to 1, and a query checks whether all k corresponding bits are set. If any bit is 0, the element is guaranteed never to have been inserted (no false negatives are possible). If all k bits are 1, the element is only possibly present, since unrelated elements may have collectively set the same bits, that is a false positive.

Kirsch–Mitzenmacher double hashing[1]: Rather than instantiating k independent hash functions, the Bloom filter implementation derives all k bit positions from just two base hashes computed via MurmurHash3 (mmh3)[2] :

$$h_i(x) = (h_1(x) + i \cdot h_2(x)) \bmod m \quad \text{for } i = 0, 1, \dots, k - 1$$

This construction is provably as effective as k truly independent uniform hash functions for Bloom filter purposes, while costing only two hash computations per element regardless of how large k is.

False-positive rate and optimal sizing: After n elements have been inserted into an m -bit array using k hash functions, the probability that a query for an absent element is incorrectly reported as present is approximated by:

$$p \approx (1 - e^{(-kn/m)})^k$$

For a target false-positive rate p and an expected element count n , the bit-array size(m) and hash-function count(k) that minimize p are given in closed form by:

$$m = - (n \cdot \ln p) / (\ln 2)^2$$

$$k = (m / n) \cdot \ln 2$$

These formulas are the basis for both the Bloom filter's default sizing and the validation performed in Section 4.4 [1].

2.0 Methodology & Setup

Both structures were implemented from scratch in Python, exposing an identical insert/query interface so benchmarking code could treat them interchangeably. Four experiments were run against two data sources:

- **Synthetic keys:** unique, randomly generated alphanumeric strings (8–24 characters), generated deterministically at scales of 10^3 , 10^4 , 10^5 , and 10^6 elements.
- **Real-world keys:** a deduplicated pool of phishing/legitimate URLs combined from three public sources.

2.1 Real-world dataset composition

Source	Description	Role in Combined pool
PhishStorm [3]	Phishing vs. legitimate URL dataset	Contributes unique URL s to the pool
PhiUSIIL [4]	Phishing URL dataset with extracted features	Contributes unique URL s to the pool
Mendeley Data [5]	Phishing URL dataset	Contributes unique URL s to the pool
Combined, deduplicated pool	Union of the three sources, shuffled	831, 866

2.2 Detailed Experimental Procedure

2.2.1 Data Generation

Synthetic keys:

1. A `random.Random(seed)` instance is created so every run with the same seed produces an identical key set, this makes the benchmark reproducible across machines.
2. An alphabet of `ascii_letters + digits` (62 characters) is fixed.
3. The generator loops: on each iteration it draws a random length between 8 and 24, then draws that many characters from the alphabet to form a candidate string, and adds it to a Python `set`.
4. Because it is a `set`, any accidental duplicate is silently absorbed rather than counted twice; the loop continues (up to $n \times 20$ attempts, a generous safety margin) until exactly n unique strings have been collected.
5. The set is converted to a list and returned.

A `parallel` function, `generate_disjoint_absent_keys(present_keys, n, seed=1337)`, generates a second batch of random strings the same way but checks each candidate against the *present* set before accepting it, guaranteeing the "absent" query set shares no element with the "present" set. This disjointness is what makes the false-positive-rate measurement meaningful: any positive result on this set is unambiguously a false positive, never a true positive in disguise.

Real-world keys (`data_loader.py`):

1. For each of the three datasets: the URL column is read, missing values are dropped, everything is coerced to a string and duplicates are removed in that single dataset.
2. `build_combined_real_url_pool(target_size=1_000_000, seed=7)` then takes the union of every loaded dataset's URLs into a single Python `set`. This is the step that deduplicates across the three sources (so a URL present in both PhishStorm and PhiUSIIL is only counted once). The union is converted to a list, shuffled with a seeded `random.Random(7)` for reproducibility, and truncated to `target_size` elements. If the deduplicated pool is smaller than the target, the function prints how many unique URLs it actually found and returns all of them rather than fabricating extras.

2.2.2 The Hashing Mechanism

Every operation on both structures ultimately calls into `hashing.py`, which wraps `MurmurHash3` via the `mmh3` library[2].

Base hashes:

1. `mmh3.hash64(item, seed=0, signed=False)` is computed and its first 64-bit component is taken as `h1`.
2. `mmh3.hash64(item, seed=0x9747b28c, signed=False)` is computed the same way, using a *different* seed, to produce `h2`. Using two different seeds of the same hash family is what makes `h1` and `h2` behave as if they were independent hash functions, without the cost of running two different hashing algorithms.
3. If `h2` happens to be exactly 0 (astronomically unlikely, but checked for safety), it is nudged to 1. A zero `h2` would collapse the double-hashing formula below into $h_i(x) = h1(x)$ for every `i`, destroying independence between the `k` positions.

Deriving `k` positions for the Bloom filter :

- Given `h1` and `h2` from the step above, the `k` bit positions are computed as:

$$position_i = (h1 + i * h2) \bmod m, \text{ for } i = 0, 1, \dots, k - 1$$

- This is the Kirsch–Mitzenmacher double-hashing construction[1]. It costs exactly two `MurmurHash3` evaluations no matter how large `k` is, the remaining `k` positions are cheap integer arithmetic, not additional hash computations.

Single hash for the Hash Set's buckets:

- A single `mmh3.hash64(item, seed=0)` value, taken modulo the current number of buckets, decides which bucket an element belongs to. This is deliberately a different, simpler mechanism than the Bloom filter's double hashing, since the hash set only ever needs one bucket index per element, not `k` of them.

2.2.3 Hash Set (V1) Operation, Step by Step

Insert(item):

1. Compute the 64-bit MurmurHash3 value for the given string, and take the modulo by number of buckets to make sure that the returned integer is within the array `length(single_hash_index)`
2. Look inside `buckets[idx]` (a plain Python list), if `item` is already in that list, do nothing and report "already present."
3. Otherwise, append `item` to `buckets[idx]` and increment the element counter.
4. Check the load factor (`count / n_buckets`). If it has exceeded 0.75, trigger a resize: allocate a new bucket array of double the size, then re-insert (rehash) every existing element into the new array using the same `single_hash_index` function against the new `n_buckets`. This is why insertion cost is only amortized $O(1)$, most insertions are cheap, but occasional insertions trigger an $O(n)$ rehash of the whole table.

Query(item):

1. Compute `idx = single_hash_index(item, n_buckets)`.
2. Check whether `item` appears in the list `buckets[idx]` via a direct linear scan and string comparison. Because the load factor is capped at 0.75, this list is short in practice, so the scan is fast but it is still a real comparison against real stored data, which is why the result is always exact.

2.2.4 Bloom Filter (V2) Operation, Step by Step

Insert(item):

1. Calculate two base hash values via MurmurHash3. Then derive `k` hash values as explained above in section 2.2.2 [Final Report](#) (`derive_positions(item, k, m)`)
2. Set each of those `k` bits in the underlying `bitarray` to 1 (bits already at 1 are simply left as 1 insertion never "un-sets" anything).
3. Increment an internal insert counter (used only for reporting; it is not a count of unique elements, since inserting the same element twice sets the same bits again with no way to detect the repeat).

Query(item):

1. Call `derive_positions(item, k, m)` to get the same `k` positions that would have been set had this exact item been inserted.

2. Check every one of those k bits. If any bit is 0, return `False`. This element is guaranteed never to have been inserted, because insertion can only ever set bits to 1, never clear them. If all bits are 1, return `True` meaning "possibly inserted," since those bits could have been set by some combination of other elements.

2.2.5 Benchmark Procedure

Memory footprint , for each target size n :

1. Generate n keys via the active key generator (synthetic or real).
2. Build a fresh `HashSet` while `tracemalloc` is active, recording the peak Python-level allocation reached during the build.
3. Build a second, independent `HashSet` with the same keys while measuring the process's resident-set-size (RSS) delta via `psutil`, as a real-world cross-check.
4. Size a `BloomFilter` for this n using `BloomFilter.from_target_fpr(n, target_fpr=0.01)` this internally computes m^* and k^* from the formulas in Section 1, then repeat the same `tracemalloc`/RSS measurement pair for it.
5. Record the Bloom filter's exact size as well, computed directly as $m / 8$ bytes (the literal size of its bit array), since this is a closed-form value that needs no measurement.
6. A `primary_bytes` column is populated with the most reliable number available for each structure (`tracemalloc` peak for the hash set, the exact $m/8$ value for the Bloom filter), because raw RSS deltas are dominated by OS page-granularity (4 KB) noise for the Bloom filter's naturally small allocations.

Latency, for each target size n :

1. Generate n present keys and an equal-sized disjoint set of absent keys.
2. Insertion timing: rebuild a brand-new (empty) structure and time inserting all n keys, one at a time, using `time.perf_counter()` before and after the loop. This is repeated `n_trials=10` times, each on a fresh structure, so that later trials are never measuring inserts into an already-populated (and possibly already-resized) table.
3. Query timing: build one structure containing all n keys, then separately time querying the full present-key sample and the full absent-key sample, each repeated `n_trials` times against the same pre-built structure (query timing does not rebuild the structure, since queries shouldn't mutate it).
4. Every timing set is summarized (mean, median, standard deviation, min, max) and converted to microseconds-per-operation by dividing the total wall-clock time by the number of operations performed.

False-positive rate , for a *fixed* (m, k) pair:

1. Generate the full key pool once (sized to the largest checkpoint requested) and a large disjoint absent-key set (default 20,000 keys), generated once and reused at every checkpoint for consistency.
2. Starting from an empty Bloom filter, walk through the checkpoints in increasing order. At each checkpoint, insert only the new keys since the previous checkpoint (so the filter accumulates state rather than being rebuilt).
3. At each checkpoint:
 - a. Query every key inserted so far and confirm all return `True`. This is the explicit zero-false-negative check
 - b. Query the entire fixed absent-key set and count how many incorrectly return `True`. Dividing this count by the absent-set size gives the empirical FPR.
 - c. Compute the theoretical FPR from the formula in Section 1 using the current cumulative n .
 - d. Record the fill ratio (fraction of bits set to 1) alongside both FPR values for comparison.

Parameter sensitivity , for a fixed n :

1. Generate the n keys and one disjoint absent-key set (default 10,000 keys) once, reused across the entire sweep.
2. For every combination of m in the configured grid (bit-array sizes corresponding to 4, 8, 12, 16, and 20 bits per element) and k in the configured grid ($\{1, 2, 3, 4, 5, 6, 7, 8, 10, 12\}$), build a brand-new `BloomFilter( $m, k$ )`, insert all n keys into it, and query the fixed absent set to compute its empirical FPR.
3. For each row, also compute the analytically optimal $k^* = (m/n) \cdot \ln 2$ for that m , so the empirical results can be compared directly against theory afterward.
4. `find_best_k_per_m` then groups the results by m and, within each group, picks the row with the lowest empirical FPR, this is the "best empirical k " reported in Section 4.4, compared against the theoretical k^* computed in step 3.

2.2.6 Orchestration

`scripts/run_benchmarks.py` ties all four benchmarks together for a given data source (synthetic, real, or both). In `--full` mode; the mode used to produce the attached results , it runs the memory and latency benchmarks at $n \in \{1,000; 10,000; 100,000; 800,000\}$, the FPR benchmark at cumulative checkpoints up to 800,000 with m fixed at 8 bits $\times$ the largest checkpoint and $k = 6$, and the parameter sweep at a fixed $n = 100,000$ across the (m, k) grid described above. For the real-data run, every size list is automatically filtered down to values no larger than the actual combined URL pool, which is why the real-data tables stop at 100,000 (memory/latency) and 500,000 (FPR) rather than reaching 1,000,000.

2.3 Environment Setup

Both bitarray and mmh3 ship prebuilt wheels for common platforms, so neither uv nor pip should need to compile anything from source under normal conditions.

(01) Windows

Option A: uv

```
uv sync
```

Option B: pip

```
python -m venv .venv

.venv\Scripts\activate

pip install -r requirements.txt

pip install -e .
```

macOS

Option A: uv

```
uv sync
```

Option B: pip

```
python3 -m venv .venv

source .venv/bin/activate

pip install -r requirements.txt

pip install -e .
```

2.4 Benchmark execution

```
uv run scripts/run_benchmarks.py --full --mode both
```

This produces the CSV tables and PNG plots analyzed below, targeting ‘results/tables/’ and ‘results/plots/’ respectively.

3.0 Results and Discussion

3.1 Memory Footprint

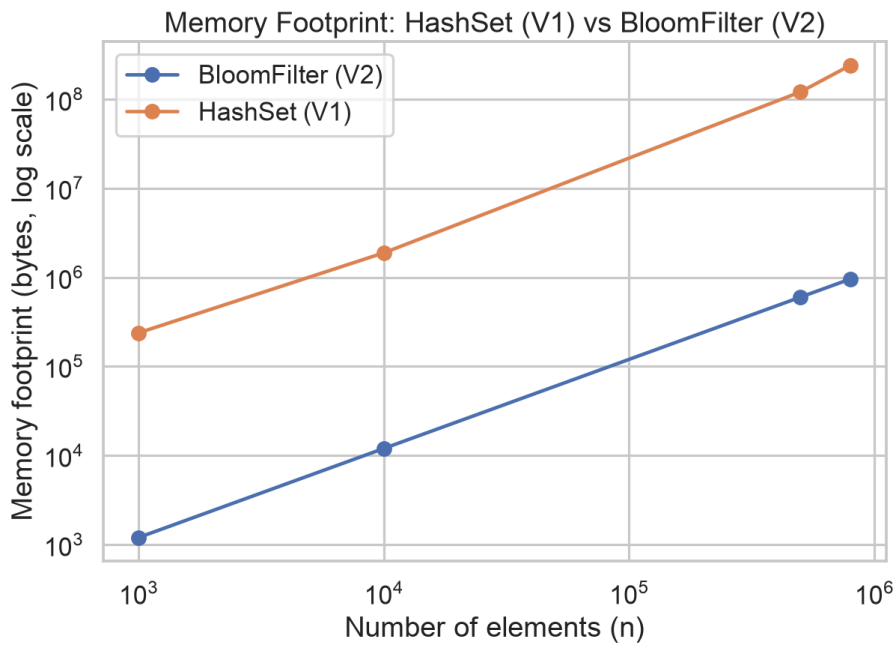


Figure 1 — Memory footprint (log-log): HashSet (V1) vs. BloomFilter (V2), synthetic keys.

n	HashSet (V1)	BloomFilter (V2)	Reduction
1,000	236.95 KB	1.20 KB	197.6×
10,000	1.88 MB	11.98 KB	156.9×
500,000	120.59 MB	599.07 KB	201.3×
800,000	240.49 MB	958.51 KB	250.9×

Table 1 - Memory footprint, synthetic keys

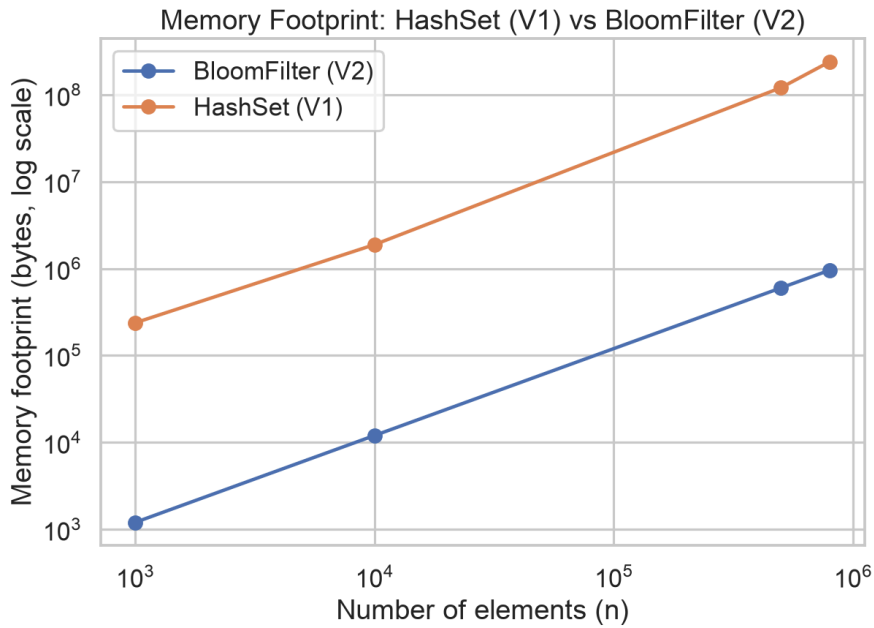


Figure 2 — Memory footprint (log-log): HashSet (V1) vs. BloomFilter (V2), real-world phishing URLs

n	HashSet (V1)	BloomFilter (V2)	Reduction
1,000	237.37 KB	1.20 KB	198.0×
10,000	1.88 MB	11.98 KB	157.2×
500,000	120.61 MB	599.07 KB	201.3×
800,000	240.47 MB	958.51 KB	250.9×

Table 2 - Memory footprint, real-world phishing URLs

(HashSet figures are ‘tracemalloc’ peak allocations; BloomFilter figures are the exact closed-form size $m/8$ bytes.)

Both structures scale roughly linearly with ‘n’ on the log-log plot, but the vertical separation between the two lines are more than two orders of magnitude at every scale tested. This is explained by what each byte is spent on rather than by asymptotic order. The Bloom filter consistently costs **9.585 bits per element** regardless of ‘n’. The hash set, by contrast, must retain every string in full: with CPython's per-object overhead, the observed cost is roughly **1,500–2,400 bits per element** two to three orders of magnitude more per element than the Bloom filter.

The reason for this is because the Hashset stores the string in memory, while the Bloom Filter stores no per-element data at all, only a shared bit array whose individual bits carry no information about which elements set them.

3.2 Latency Analysis

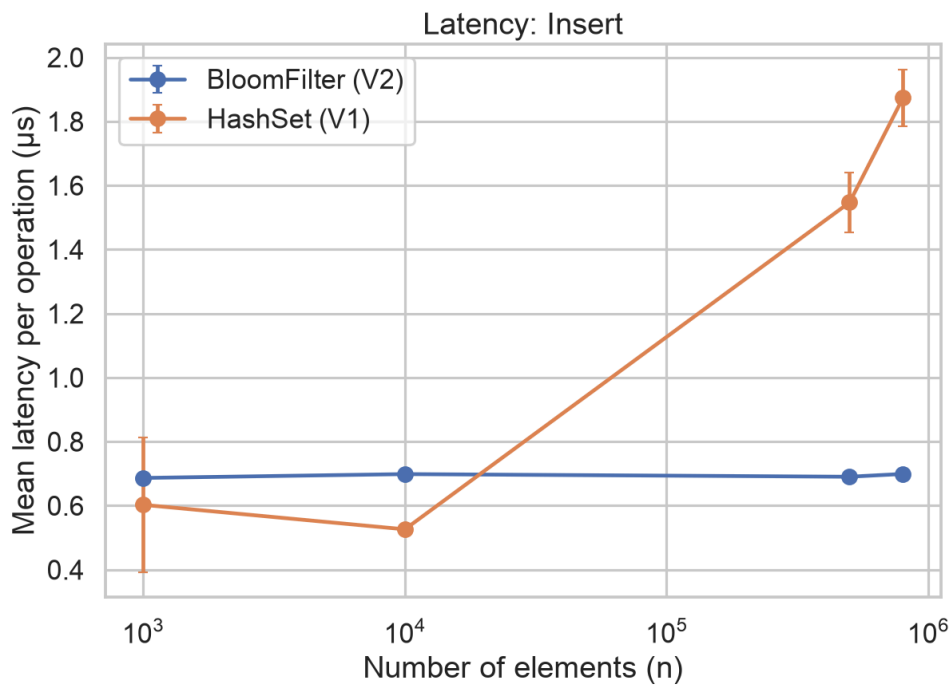


Figure 3 - Mean insertion latency per operation, synthetic keys.

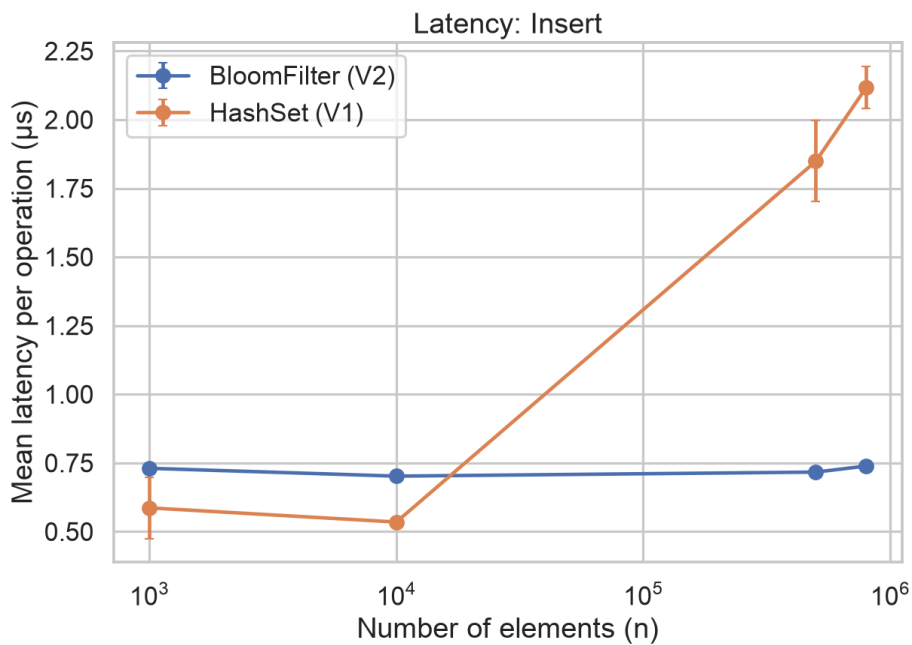


Figure 4 - Mean insertion latency per operation, real-world phishing URLs.

Table 3- Insertion latency (μs/op)

n	HashSet (synthetic)	BloomFilter (synthetic)	HashSet (real)	BloomFilter (real)
1,000	0.60	0.69	0.59	0.73

10,000	0.53	0.70	0.53	0.70
500,000	1.55	0.69	1.85	0.72
800,000	1.87	0.70	2.12	0.74

The Bloom filter's insertion cost is essentially flat across three orders of magnitude of 'n' (0.73-0.74 $\mu\text{s/op}$), exactly as the ' $O(k)$ ' complexity predicts. The hash set's insertion cost climbs from 0.59 $\mu\text{s/op}$ at

$n = 10,000$ to 2.12 $\mu\text{s/op}$ at $n = 800,000$.

The reason for this is because in the Bloom Filter, no matter how the number of insertion items increases it still does the same hashing only. There will not be any added overhead with the increase of the number of keys. But in the Hashset, with the increase of number of keys the possibility of hash collisions increases. Therefore the string s then has to be stored in the hashset at the end of the linked list in that particular basket obtained by hashing. This additional overhead is what causes the increase of insertion time in Hashset with the increase of number of keys.

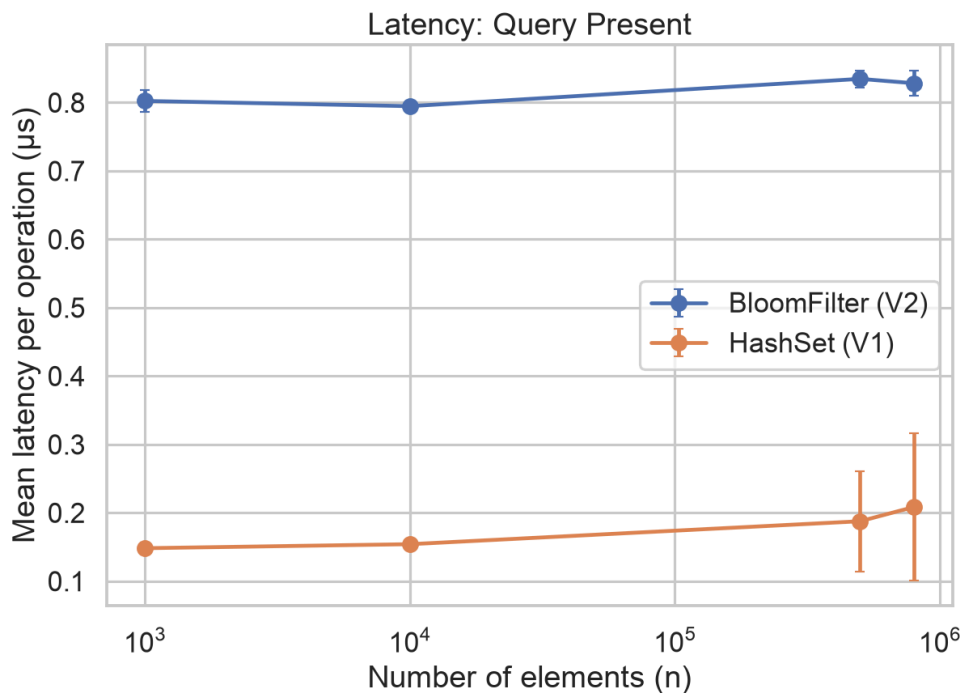


Figure 5 — Mean query latency per operation for present elements, synthetic keys.

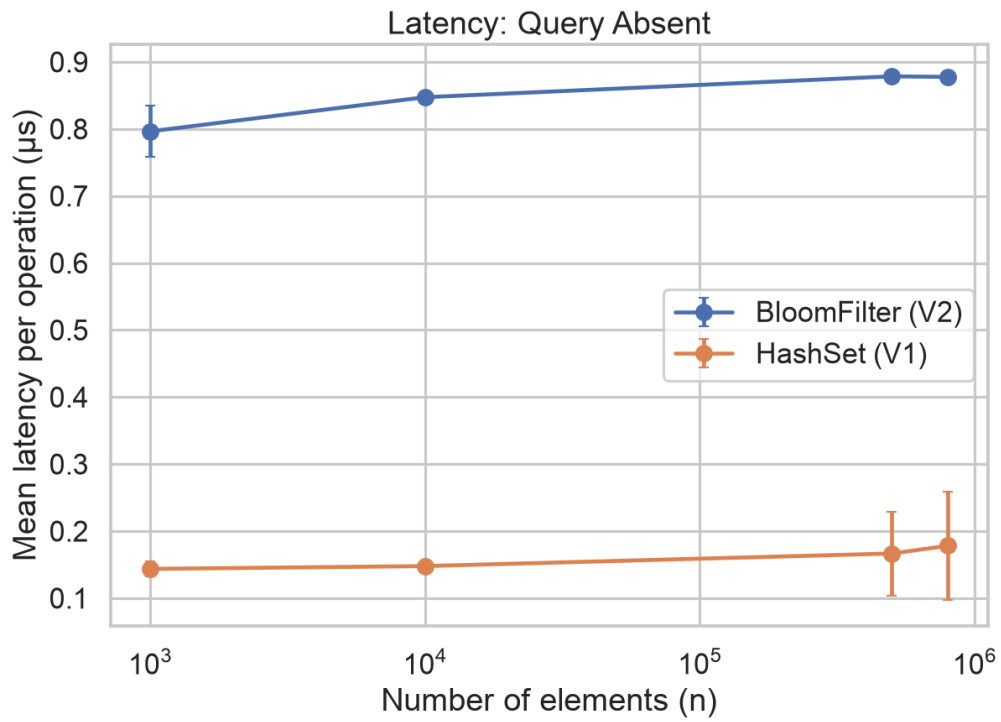


Figure 6 - Mean query latency per operation for absent elements, synthetic keys.

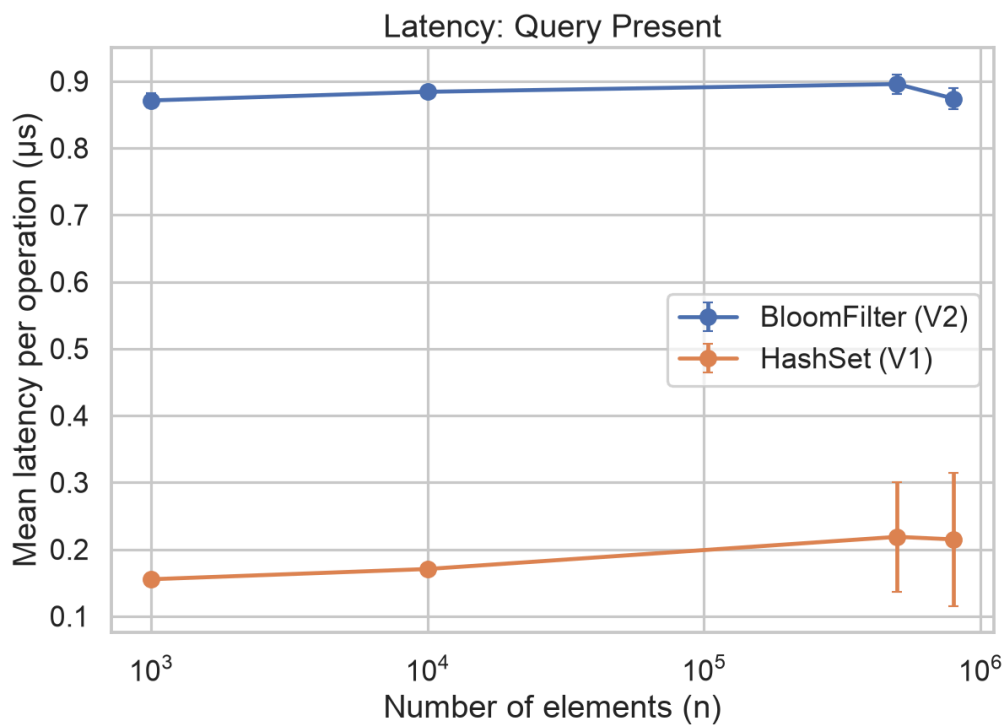


Figure 7 - Mean query latency per operation for present elements, real keys.

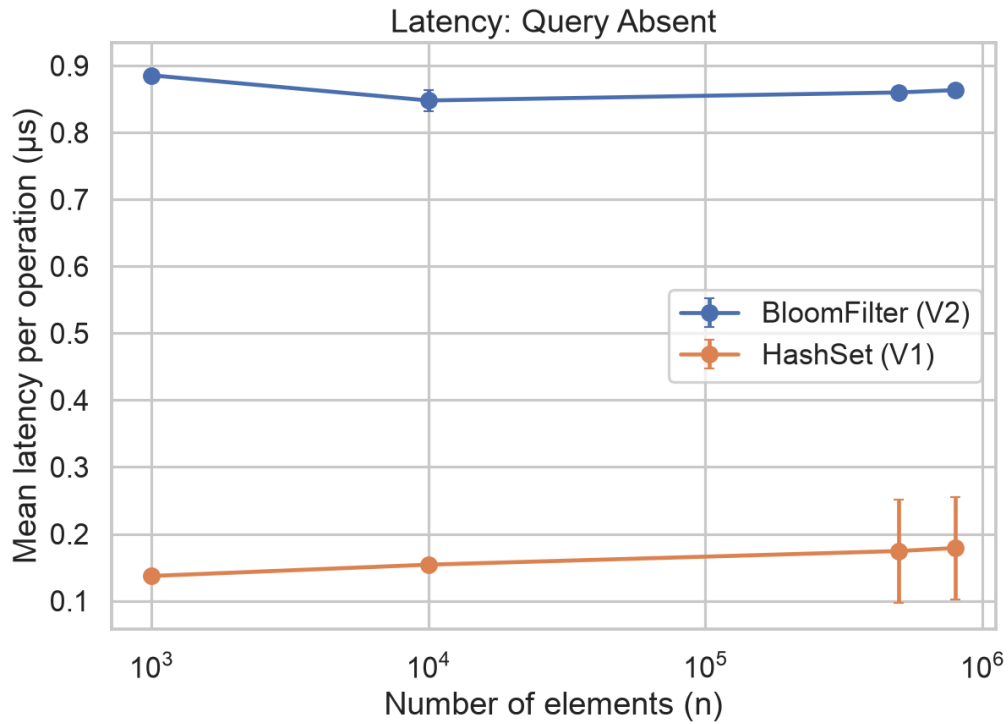


Figure 8 - Mean query latency per operation for absent elements, real keys.

Table 4 - Query latency, present vs. absent (μs/op, synthetic keys)

n	HashSet (present)	HashSet (absent)	BloomFilter (present)	BloomFilter (absent)
1,000	0.15	0.14	0.80	0.80
10,000	0.15	0.15	0.79	0.85
500,000	0.19	0.17	0.83	0.88
800,000	0.21	0.18	0.83	0.88

Table 5 - Query latency, present vs. absent (μs/op, real-world URLs)

n	HashSet (present)	HashSet (absent)	BloomFilter (present)	BloomFilter (absent)
1,000	0.16	0.14	0.87	0.89
10,000	0.17	0.15	0.88	0.85
500,000	0.22	0.17	0.90	0.86
800,000	0.21	0.18	0.87	0.86

Interestingly, queries invert the pattern seen for insertion: the hash set is 4–6× faster per query than the Bloom filter throughout. A Bloom filter query must compute two MurmurHash3 hashes and then perform k separate bit-array index operations, carrying Python-level call overhead; that fixed constant is simply larger than the hash set's typical one-or-two comparisons at these load factors. Also, the implementation being done in Python is a contributing factor. Since the hashset has only one highly optimised function(calculating the mmh3 hash) which is implemented underhood in C. But Bloom Filter has to use the two similarly computed hashes and create k hashes, and iterate through the bitarray, which is all implemented in Python. At the load factor and key sizes tested here, the hash set's per-query constant is simply small enough that the Bloom filter's extra hash computation and per- k -iteration interpreter overhead dominates. A lower-level language implementation (C/C++/Rust) with hash-inlined loops would likely narrow or reverse this gap, since the interpreter round-trip cost that's driving the difference here is largely a C-Python-specific overhead, not a fundamental property of either data structure.

3.3 False-Positive Rate (FPR)

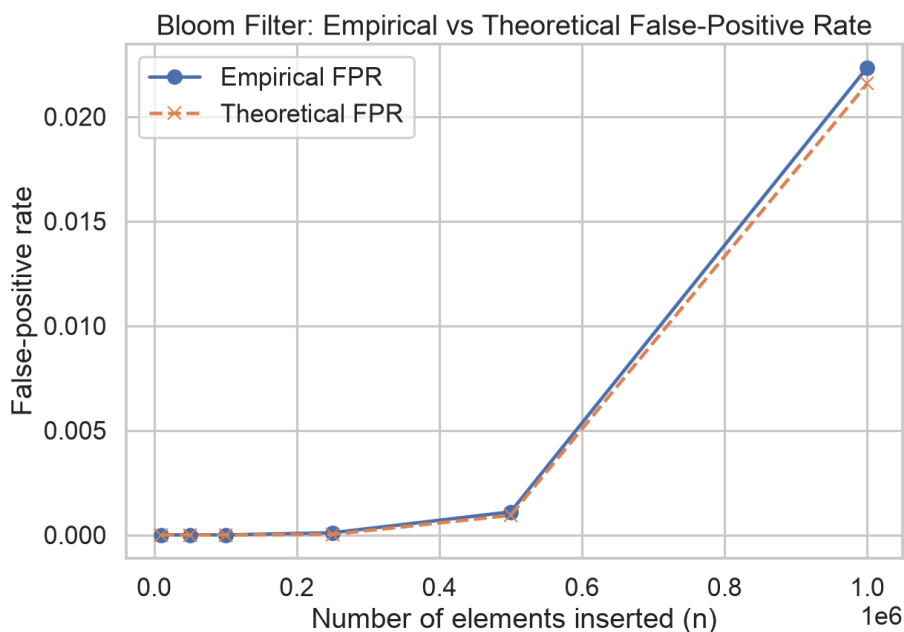


Figure 9 - Empirical vs. theoretical false-positive rate as 'n' grows, fixed 'm = 8,000,000', 'k = 6' (synthetic keys).

Table 6 - Empirical vs. theoretical FPR, synthetic keys (m = 8,000,000, k = 6)

n	Fill ratio	Empirical FPR	Theoretical FPR	Absolute error
10,000	0.75%	0.0000	1.74×10^{-13}	~ 0
50,000	3.68%	0.0000	2.49×10^{-9}	~ 0

100,000	7.23%	0.0000	1.42×10^{-7}	~ 0
250,000	17.10%	0.0001	2.50×10^{-5}	7.50×10^{-5}
500,000	31.27%	0.0011	0.000935	1.65×10^{-4}
1,000,000	52.77%	0.02235	0.02158	7.73×10^{-4}

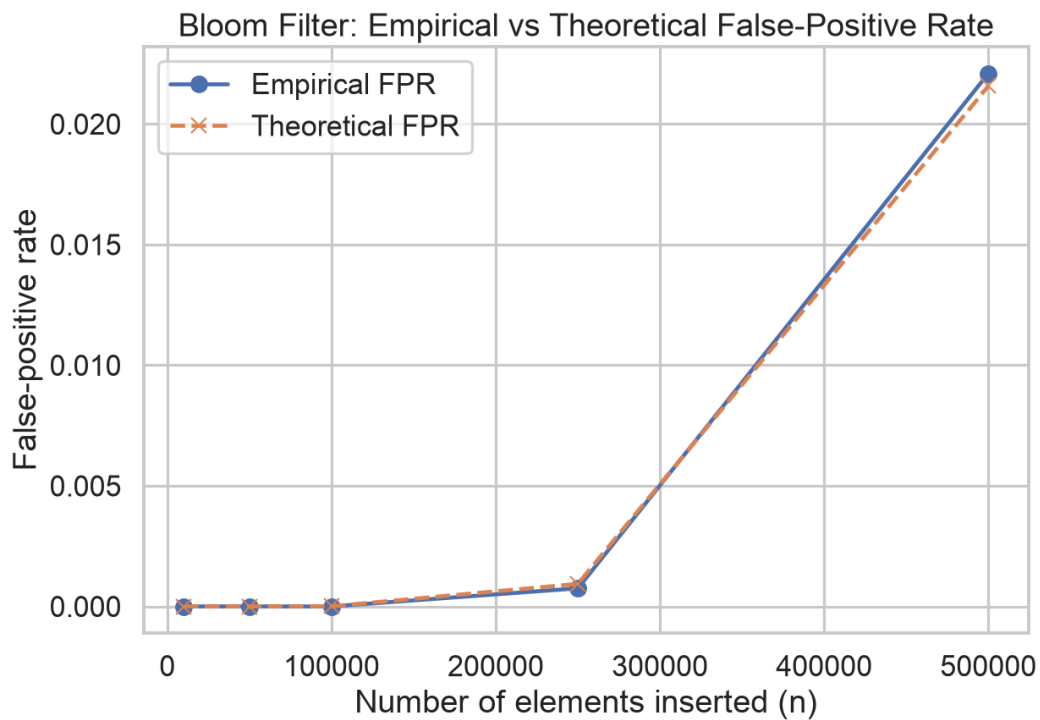


Figure 10 — Empirical vs. theoretical false-positive rate, real-world phishing URLs ($m = 4,000,000$, $k = 6$).

Table 7 - Empirical vs. theoretical FPR, real-world URLs ($m = 4,000,000$, $k = 6$)

n	Fill ratio	Empirical FPR	Theoretical FPR	Absolute error
10,000	1.49%	0.0000	1.09×10^{-11}	~ 0
50,000	7.23%	0.0000	1.42×10^{-7}	~ 0
100,000	13.93%	0.0000	7.30×10^{-6}	~ 0
250,000	31.27%	0.00075	0.000935	1.85×10^{-4}
500,000	52.79%	0.0221	0.02158	5.23×10^{-4}

A false positive rate is only discussed for the Bloom Filter, since the hashset will not have a false positive rate. As for the Bloom filter, the empirical curve tracks the theoretical curve extremely closely. Also, `zero_false_negatives_holds` was True at every single checkpoint, confirming that the Bloom filter never once failed to recognize a previously inserted element. The equation used to calculate the theoretical FPR is given under section 1.

3.4 Parameter Sensitivity

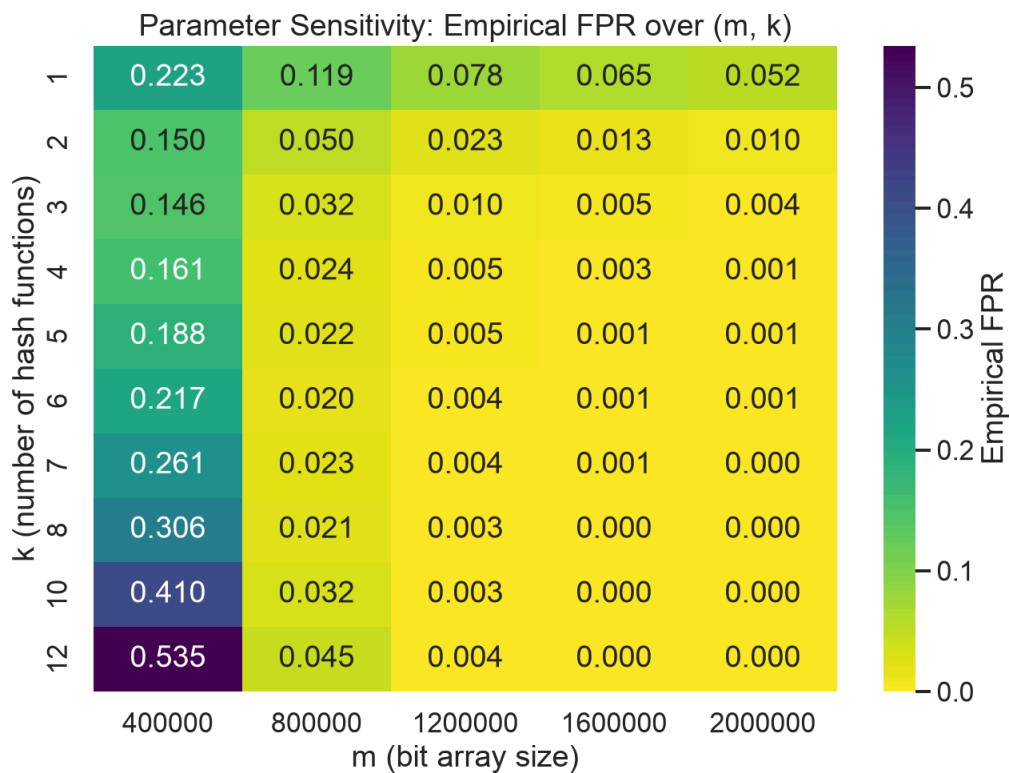


Figure 11 - Empirical false-positive rate for synthetic data over the full `(m, k)` grid at fixed `n = 100,000`.

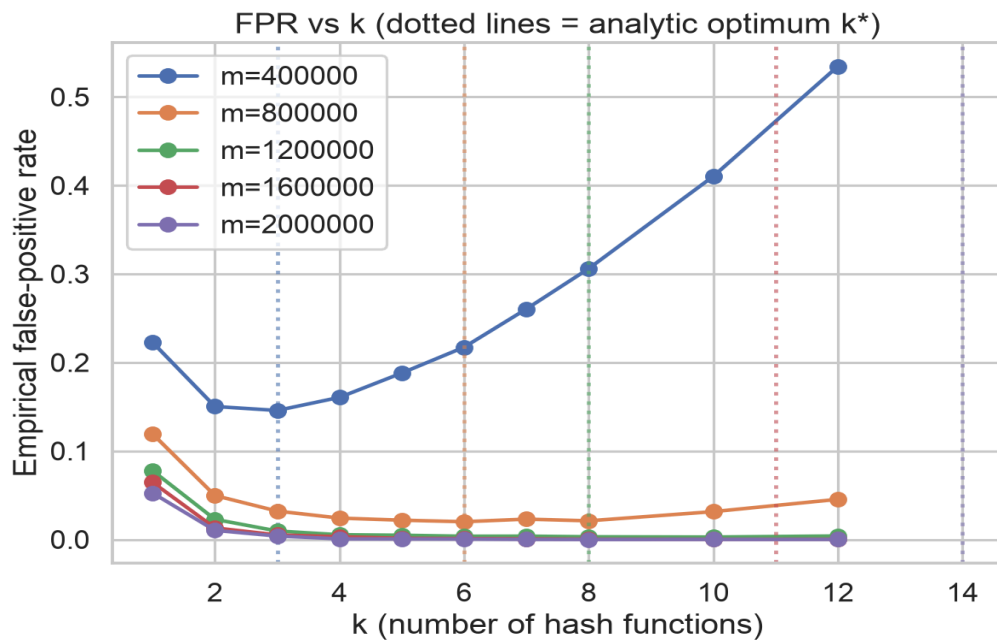


Figure 12 - Empirical FPR vs. 'k' for each 'm' for synthetic data ; dotted vertical lines mark the analytically predicted optimum ' $k = (m/n) \cdot \ln 2$ '.

Table 8 - Best empirical k per m, synthetic keys (fixed $n = 100,000$)

m	Bits/element	k^* (analytic)	Best empirical k	Empirical FPR at best k	Theoretical FPR at best k	Match?
400,000	4.0	3	3	0.1458	0.1469	Exact
800,000	8.0	6	6	0.0201	0.0216	Exact
1,200,000	12.0	8	10	0.0028	0.0033	$\approx$ Adjacent
1,600,000	16.0	11	10	0.0003	0.0005	$\approx$ Adjacent
2,000,000	20.0	14	8	0.0000	0.0001	$\approx$ Grid-limited

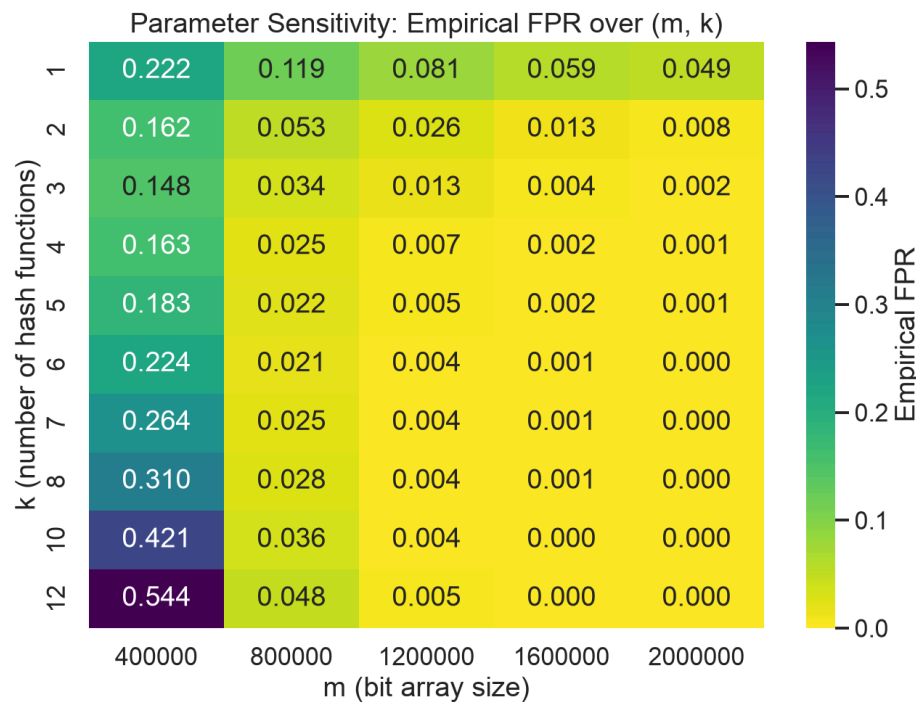


Figure 13 - Empirical false-positive rate for real data over the full `(m, k)` grid at fixed `n = 100,000`.

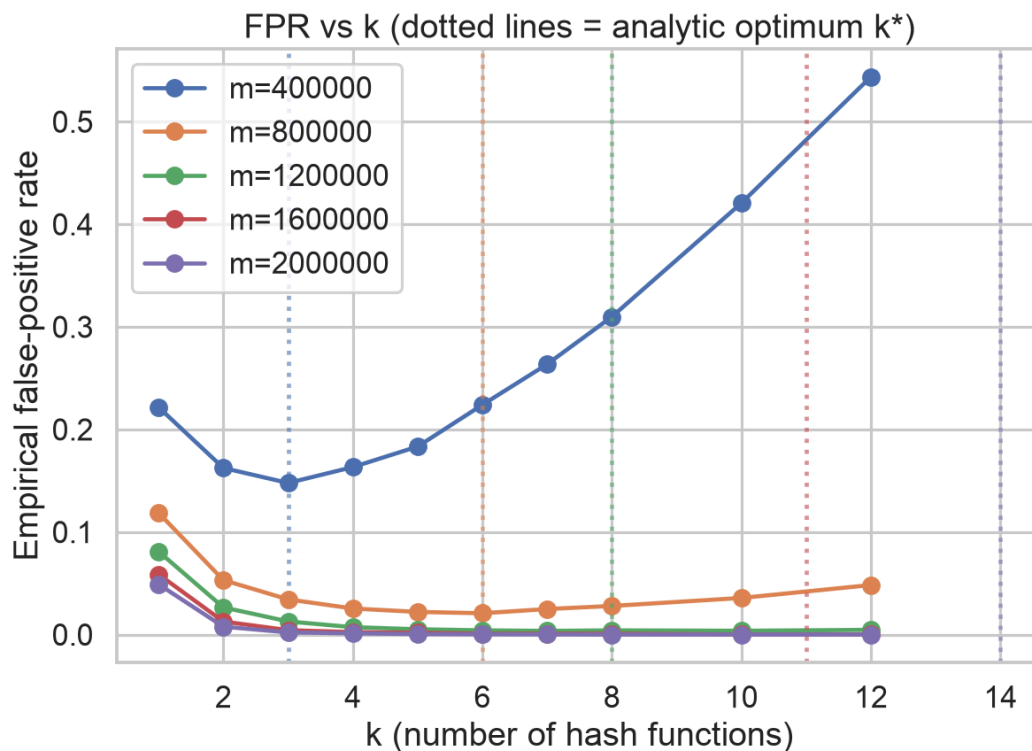


Figure 14 - Empirical FPR vs. `k` for each `m` for real data ; dotted vertical lines mark the analytically predicted optimum $k^* = (m/n) \cdot \ln 2$.

Table 9 - Best empirical k per m, real-world URLs (fixed n = 100,000)

m	Bits/element	k* (analytic)	Best empirical k	Empirical FPR at best k	Theoretical FPR at best k	Match?
400,000	4.0	3	3	0.1478	0.1469	Exact
800,000	8.0	6	6	0.0208	0.0216	Exact
1,200,000	12.0	8	7	0.0036	0.0033	≈ Adjacent
1,600,000	16.0	11	10	0.0001	0.0005	≈ Adjacent
2,000,000	20.0	14	8	0.0000	0.0001	≈ Grid-limited

For every value of ‘m’, sweeping ‘k’ produces the U-shaped curve theory predicts. At lower ‘m’ values, the empirically best-performing ‘k’ matches the analytically predicted ‘k’ exactly. As ‘m’ increases this confirmation of the theoretical ‘k’ with the empirical value starts to differ.

This deviation can be justified by the following factors:

(01) Grid-Resolution Artifacts at High Capacities:

At lower array sizes (like m=400 000), the empirically optimal k aligns seamlessly with the theoretical prediction. However, at m=1,600,000 and m=2,000,000, the empirical optimum appears to deviate from the theoretical k. This divergence is not a failure of the probabilistic model, but rather an artifact of the experimental grid resolution. The parameter sweep capped the maximum hash-function count at k=12. Because the theoretical optimums for these sizes are k=11 and k=14 respectively, the true analytical optimum for m=2,000,000 resided entirely outside the tested search space, making a direct empirical match impossible within this specific benchmark run.

(02)Statistical Noise and the Law of Small Numbers

This deviation is also majorly driven by sampling noise. When evaluating the FPR at high capacities, the expected error rate drops to fractions of a percent. When translated into raw counts against a fixed absent-query sample of 10,000 items, an FPR of 0.01\% represents just a single false positive. At m=400,000, false-positive counts are in the hundreds or thousands, yielding a statistically robust optimum. Conversely, at m=1,200,000 and above, the raw false-positive counts drop to single digits or zero. So at these high m values, a difference of one or two random hash collisions drastically skews the empirical FPR percentage.

(03)Empirical Divergence Between Datasets

This justification can also be confirmed by cross-checking the independent runs of the synthetic and real-world datasets. Under identical experimental conditions at m=1,200,000, the synthetic dataset identified k=10 as the optimal configuration, while the real-world dataset identified k=7. If the data

structure's behavior at this scale were governed purely by a reproducible analytical optimum, these independent runs would converge. This contrast in the two values confirm that at extreme values of m , the empirically "best" k is heavily dominated by random sampling noise rather than a statistically significant structural advantage.

4.0 Conclusion

The experimental results validate every core theoretical claim. The hash set delivers exact, retrievable membership at a memory cost that scales with both element count and element size. In this study, roughly 1,500–2,400 bits per stored key. The Bloom filter delivers approximate membership at a fixed, predictable cost of under 10 bits per element regardless of key size.

4.1 Recommended use cases:

Hash Set (V1) is the right choice whenever the application needs to retrieve, iterate, or delete elements, or whenever an incorrect "possibly present" answer is unacceptable.

Bloom Filter (V2) is the right choice whenever the application only needs a fast, memory-constrained pre-filter that can tolerate a small, tunable false-positive rate in exchange for a dramatic (~ 150 – $250\times$) reduction in memory.

The two structures are complementary tools: "is this exactly true?" versus "is this probably true, cheaply?" and the choice should be driven by memory constraints versus error tolerance.

4.2 Practical Applications:

- Web Browsers (Safe Browsing): Used to quickly check if a URL might be malicious before performing an expensive network lookup.
- Databases (Cassandra, HBase): Used to avoid unnecessary disk reads by checking if a key could exist in an SSTable.
- Caches & CDNs: Used to check if an object might have been requested before to inform caching decisions.
- Bitcoin SPV Clients: Used to privately request only relevant transactions from full nodes.
- Distributed Systems: Used to reduce network traffic by checking if data might exist on a node before synchronization

References

- [1] [Bloom Filters - Introduction and Implementation - GeeksforGeeks](#)
- [2] Kirsch, A., & Mitzenmacher, M. (2008). Less hashing, same performance: Building a better Bloom filter. *Random Structures and Algorithms*, 33(2), 187–218. <https://doi.org/10.1002/rsa.20208>
- [3] Senuma, H. (2025). mmh3: A Python extension for MurmurHash3. *Journal of Open Source Software*, 10(105), 6124. <https://doi.org/10.21105/joss.06124>
- [4] <https://research.aalto.fi/en/datasets/phishstorm-phishing-legitimate-url-dataset/>
- [5] <https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset>
- [6] <https://data.mendeley.com/datasets/vfszbj9b36/1>

AI Tools Used

- [1] Claude
- [2] Gemini